FizzShare — Project Proposal

Project Proposal for Exhibition Submission

Project Title: FizzShare — Peer-to-Peer File Sharing Without the Middleman **Prepared by:** Irfan Ali **Date:** September 19, 2026

---

1. Introduction

FizzShare is a **peer-to-peer (P2P) file-sharing web application** that lets two people transfer files of any size directly between their devices — phone, laptop, or desktop — using nothing but a browser. There is **no cloud upload, no account, no installation**. The sender selects files, a share link with a short code is created, and the recipient opens the link on any device to instantly download the files.

The experience is deliberately modeled after ToffeeShare (toffeeshare.com), a real-world reference for a clean, friendly, "sweet" interface.

2. Problem Statement

Conventional file-sharing services have significant drawbacks:

- **Cloud upload+download is slow** — files are uploaded to a server first, then downloaded again, wasting bandwidth and time.
- **File-size limits** — most free services cap transfers (e.g., 2 GB, 100 MB).
- **Privacy concerns** — files pass through and are stored (even temporarily) on third-party servers.
- **Installation burden** — desktop apps often require software installs; USB/Flash drives fail in cross-platform and remote scenarios.

3. Objectives

1. Share files of **any size** directly between browsers.
2. Ensure **end-to-end encryption** so file bytes never touch a cloud server.
3. Make sharing as fast and frictionless as possible (no accounts, no installs, no file-size limits).
4. Deliver a polished, exhibition-ready web app on a public HTTPS link that works on mobile phones.

4. Proposed Solution & How It Works

FizzShare uses **WebRTC (RTCPeerConnection + RTCDataChannel)** for a true peer-to-peer data path. The architecture has three parts running in Docker:

```
+-----------+ +----------------+ +-----------+ | Sender | ========== | WebSocket | ========== | Receiver | | browser | P2P | signaling | P2P | browser | | (file) |<==========>| + STUN/TURN | <==========>| (file) | +-----------+ WebRTC | (coturn) | WebRTC +-----------+ +----------------+
```

- **Signaling server (Node.js/TypeScript + WebSocket):** coordinates the "handshake" — it creates rooms with short codes and exchanges connection details (SDP + ICE candidates), minting short-lived TURN credentials. It **never sees or stores file data**.
- **STUN/TURN (coturn):** discovers public IPs; when direct connection is blocked by a strict NAT/firewall (e.g., phone on cellular vs. laptop on office Wi-Fi), it relays traffic, keeping throughput high via a UDP port range.
- **Caddy:** automatic HTTPS certificates (Let's Encrypt) and reverse proxy — browsers run on the secure origin, so the P2P features work on mobile.
- **Browser client (TypeScript/WebRTC):** drag-and-drop file selection, streaming transfer in chunks over an ordered, reliable data channel, live progress, pause/ cancel, and automatic integrity verification via **SHA-256** on both sides.

Recovery logic handles flaky networks (chunk re-transmission), and a built-in Network Inspector shows ICE candidate paths, data rates, and round-trip time.

5. Key Features

- Direct peer-to-peer file transfer — **no file size limit**
- End-to-end transport encryption (DTLS) — server never touches file bytes
- Share via short code/link (e.g., `https://fizzshare.link/c/ABC123`)
- No accounts, no installation — works in any modern browser
- Multi-file batch sharing with live per-file progress, pause & cancel
- Automatic accept + download with on-device SHA-256 integrity verification
- Network Inspector: live ICE candidates, path type, RTT, throughput, relay toggle
- Sweet, modern UI modeled on ToffeeShare (fully responsive, dark-mode aware)
- Self-healing on unstable connections (chunked, ordered delivery)

6. Technology Stack

Layer	Technology
Client	TypeScript, WebRTC (RTCPeerConnection, RTCDataChannel), WebSocket, esbuild
Server	Node.js, TypeScript, WebSocket (clean-slate implementation)
Network	coturn (STUN/TURN), Caddy (HTTPS/ACME reverse proxy)
Deployment	Docker Compose, AWS EC2 (free tier) + Elastic IP, sslip.io + Let's Encrypt
Testing	Vitest (unit), automated two-browser CDP end-to-end harness (byte-exact SHA-256 checks)

7. Methodology & Timeline

Phase	Milestone	Status
1	Protocol & server design	Completed
2	WebRTC browser client (transfer engine, chunking, retries)	Completed
3	Client + server test suites (16 client, 28 server unit tests)	Completed
4	Containerized deployment (signaling, coturn, Caddy)	Completed
5	ToffeeShare-style UI redesign	Completed
6	Public cloud deployment (HTTPS, Elastic IP)	Completed
7	End-to-end verification incl. TURN relay path	In progress

8. Testing & Verification

- **Unit tests:** 16 client tests and 28 server tests (Vitest) covering protocol state machines, chunk framing, integrity checking, and room lifecycle.
- **End-to-end:** an automated two-browser harness (headless Chrome via the Chrome DevTools Protocol) performs a real transfer over the public internet and verifies the downloaded file is **byte-for-byte identical** (SHA-256), including a relay-only scenario that forces traffic through the TURN server (proving NAT traversal works for phones on cellular networks).

9. Exhibition-Ready Deliverable

The running app is publicly accessible with a real HTTPS certificate, so exhibition visitors can simply open the share link on their phones and download the files directly.

- Public demo endpoint: secure HTTPS page with a share link flow
- Live showcase flow: sender selects files → share link appears → visitor opens link on phone → files download directly with progress and automatic verification
- Fully offline-appropriate: works across Wi-Fi/mobile networks, no accounts required

10. Future Scope

- QR-code share links for even faster mobile handoff.
- Local discovery (mDNS/LAN) mode for same-network transfers.
- Progressive Web App (PWA) install to strengthen persistence.
- Decentralized signaling (e.g., WebRTC signaling via public relay) for 100% serverless operation.

11. Conclusion

FizzShare demonstrates a modern, secure, and surprisingly fast way to share files that cheats the classic upload-them-then-download-them pattern entirely. It brings together real-world networking concepts (NAT traversal, ICE, DTLS, congestion-aware chunking) inside a polished, product-grade web app — ready to be experienced live at the exhibition.